Fraud Detection of Online Transactions

Data 144 Final Project

Arturo Flores, Kennedy Goliman, Lena Lu, WingYeung Ma, Doanh Pham, Christy Quang, Misbah Syed

TABLE OF CONTENTS

01

BACKGROUND

Why this topic, potential stakeholders, etc.

02

EDA

Look at different aspects of the data, visualizations and understanding them

03

OUR MODELS

Coding approach, run-through of tested models

04

RESULTS

Best accuracy, limitations.

01

Background

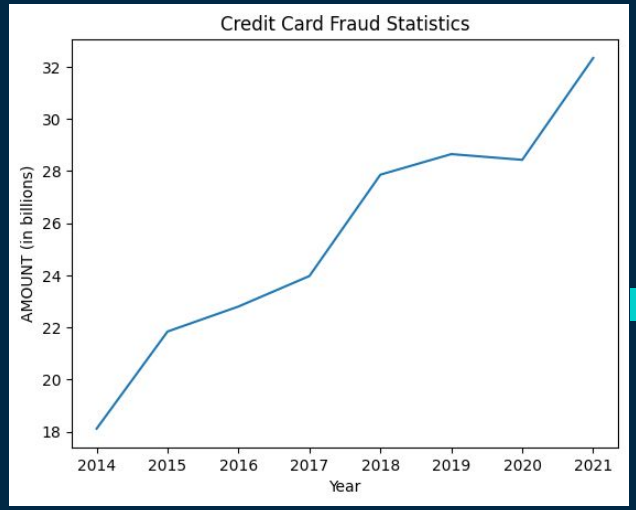
Credit Card Fraud - What is it?

- Credit card fraud is a type of identity theft and financial fraud that involves the unauthorized use of someone else's credit card information to make purchases, transactions or to access funds.
- Credit card fraud can take various forms, including:
  - Card theft
  - Skimming
  - Phishing
  - Carding
  - Account takeover
  - Data breaches
- Can result in:
  - Financial losses for individuals and businesses
  - Damage to the credit history and reputation of the affected individuals

Credit Card Fraud - Stats

- In 2021, the Federal Trade Commission (FTC) recorded nearly **390,000** reports of credit card fraud
  - One of the most common kinds of fraud in the U.S.
- In December 2022, the Nilson Report (monitors the payments industry) released a forecast indicating that U.S. losses from card fraud will total **$165.1 billion** over the next 10 years, affecting every age group in every state.
- Credit card fraud ranked as the second most common type of identity theft in the U.S.

Year	Amount (in billions)
2014	18.11
2015	21.84
2016	22.80
2017	23.97
2018	27.86
2019	28.65
2020	28.43
2021	32.34



Nilson Report, 2022

Our Objective

- Primary Objective: Develop a **predictive model** for detecting fraudulent online transactions based on details + other information of the card used
- **Machine Learning** is an ideal approach for this project
  - Able to handle large amounts of data
  - Make a quick decision in flagging a possibly fraudulent transaction (often time sensitive)
- A model can also work around the clock
  - Important when the customer base is global in nature (humans cannot work and be alert all the time)

Potential Key Stakeholders

- The model would be of great benefit to key stakeholders, such as:
  - Fraud analysts
  - Security teams
  - Credit card companies
  - Banks
  - Other decision-makers responsible for safeguarding online transactions
- These entities have it in their best interest to protect their customers
- Could help minimize financial losses due to fraudulent activities and enhance the overall security of the online transaction system

02

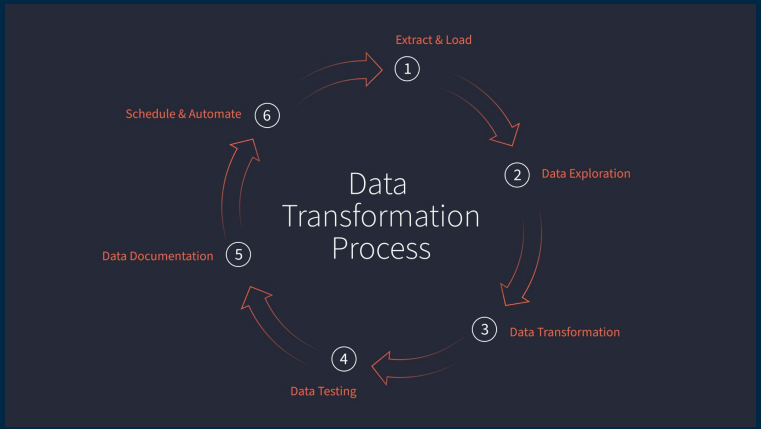
Exploratory Data Analysis

General Coding Approach Overview

- Data Exploration
  - Dataset: https://www.kaggle.com/competitions/ieee-fraud-detection/
  - Exploring the raw dataset
- Data Cleaning
  - Remove unnecessary data
  - Changing the type of data
- Data Engineering
  - Add Features
  - Remove Features
- Data Testing

	TransactionID	isFraud	TransactionDT	TransactionAmt	ProductCD	card1	card2	card3	card4	card5	...	V331
0	2987000	0	86400	68.5	W	13926	NaN	150.0	discover	142.0	...	NaN
1	2987001	0	86401	29.0	W	2755	404.0	150.0	mastercard	102.0	...	NaN
2	2987002	0	86469	59.0	W	4663	490.0	150.0	visa	166.0	...	NaN
3	2987003	0	86499	50.0	W	18132	567.0	150.0	mastercard	117.0	...	NaN
4	2987004	0	86506	50.0	H	4497	514.0	150.0	mastercard	102.0	...	0.0
5	2987005	0	86510	49.0	W	5937	555.0	150.0	visa	226.0	...	NaN
6	2987006	0	86522	159.0	W	12308	360.0	150.0	visa	166.0	...	NaN
7	2987007	0	86529	422.5	W	12695	490.0	150.0	visa	226.0	...	NaN

8 rows × 395 columns



Data Exploration

- Transactions dataset
  - `TransactionDT`: transaction time
  - `TransactionAMT`: payment amount
  - `ProductCD`: product code
  - `card1` - `card6`: payment card info
  - `c1`-`c6`: various counts (meaning masked)
  - `P_emaildomain`: purchaser email
  - `R_emaildomain`: recipient email
  - More!

- Identity dataset
  - Provides additional, supplemental information for columns in transactions
  - `DeviceInfo`: Operating system
  - `DeviceType`: Mobile or Desktop
  - More!

Data Exploration

- Data has many columns, most contain either a lot of NaN data or they don't provide meaning to the model
  - Example columns: `id31`, `id32`, `V338`, `V39`
  - Contain 90% of NaN data → security reasons

	TransactionID	isFraud	TransactionDT	TransactionAmt	ProductCD	card1	card2	card3	card4	card5	...	V331	V332
0	2987000	0	86400	68.5	W	13926	NaN	150.0	discover	142.0	...	NaN	NaN
1	2987001	0	86401	29.0	W	2755	404.0	150.0	mastercard	102.0	...	NaN	NaN
2	2987002	0	86469	59.0	W	4663	490.0	150.0	visa	166.0	...	NaN	NaN
3	2987003	0	86499	50.0	W	18132	567.0	150.0	mastercard	117.0	...	NaN	NaN
4	2987004	0	86506	50.0	H	4497	514.0	150.0	mastercard	102.0	...	0.0	0.0
5	2987005	0	86510	49.0	W	5937	555.0	150.0	visa	226.0	...	NaN	NaN
6	2987006	0	86522	159.0	W	12308	360.0	150.0	visa	166.0	...	NaN	NaN
7	2987007	0	86529	422.5	W	12695	490.0	150.0	visa	226.0	...	NaN	NaN

8 rows × 395 columns

`train_transaction.csv`

```
train_transaction.isna().sum()

TransactionID         0
isFraud               0
TransactionDT         0
TransactionAmt        0
ProductCD             0
                    ...
V336               3366
V337               3366
V338               3366
V339               3366
TransactionHour       0
```

Data Cleaning

- Removed columns that had 30% or more null data
- Removed rows that had 30% or more null data
- Moved forward with these cut down, more easily manageable datasets

Before
(lots of NaN 😞)

	TransactionID	isFraud	TransactionDT	TransactionAmt	ProductCD	card1	card2	card3	card4	card5	...	V331	V332	V333	V334	V335	V336	V337	V338	V339	TransactionHour
0	2987000	0	86400	68.5	W	13926	NaN	150.0	discover	142.0	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0
1	2987001	0	86401	29.0	W	2755	404.0	150.0	mastercard	102.0	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0
2	2987002	0	86469	59.0	W	4663	490.0	150.0	visa	166.0	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0
3	2987003	0	86499	50.0	W	18132	567.0	150.0	mastercard	117.0	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0
4	2987004	0	86506	50.0	H	4497	514.0	150.0	mastercard	102.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0
5	2987005	0	86510	49.0	W	5937	555.0	150.0	visa	226.0	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0
6	2987006	0	86522	159.0	W	12308	360.0	150.0	visa	166.0	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0
7	2987007	0	86529	422.5	W	12695	490.0	150.0	visa	226.0	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0

After
(less NaN 🥰)

	TransactionID	isFraud	TransactionDT	TransactionAmt	ProductCD	card1	card2	card3	card4	card5	...	V312	V313	V314	V315	V316	V317	V318	V319	V320	V321
0	2987000	0	86400	68.50	W	13926	NaN	150.0	discover	142.0	...	0.0	0.000000	0.000000	0.000000	0.0	117.0	0.0	0.000000	0.000000	0.000000
1	2987001	0	86401	29.00	W	2755	404.0	150.0	mastercard	102.0	...	0.0	0.000000	0.000000	0.000000	0.0	0.0	0.0	0.000000	0.000000	0.000000
2	2987002	0	86469	59.00	W	4663	490.0	150.0	visa	166.0	...	0.0	0.000000	0.000000	0.000000	0.0	0.0	0.0	0.000000	0.000000	0.000000
3	2987003	0	86499	50.00	W	18132	567.0	150.0	mastercard	117.0	...	135.0	0.000000	0.000000	0.000000	50.0	1404.0	790.0	0.000000	0.000000	0.000000
4	2987004	0	86506	50.00	H	4497	514.0	150.0	mastercard	102.0	...	0.0	0.000000	0.000000	0.000000	0.0	0.0	0.0	0.000000	0.000000	0.000000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
590535	3577535	0	15811047	49.00	W	6550	NaN	150.0	visa	226.0	...	0.0	47.950001	47.950001	47.950001	0.0	0.0	0.0	0.000000	0.000000	0.000000
590536	3577536	0	15811049	39.50	W	10444	225.0	150.0	mastercard	224.0	...	0.0	0.000000	0.000000	0.000000	0.0	0.0	0.0	0.000000	0.000000	0.000000
590537	3577537	0	15811079	30.95	W	12037	595.0	150.0	mastercard	224.0	...	0.0	0.000000	0.000000	0.000000	0.0	0.0	0.0	0.000000	0.000000	0.000000

Feature Engineering

- TransactionAmt
  - Log function
  - Removed decimal
  - Measure decimal length
- TransactionDT
  - Extract only hour and only day

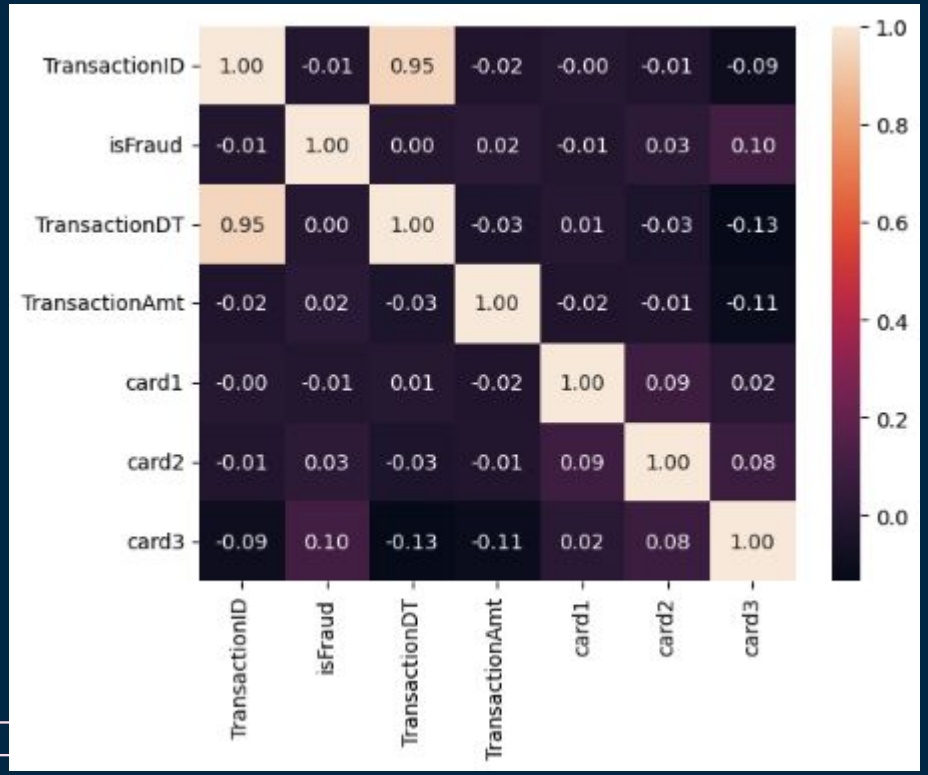
- TransactionAmt
  - Normalized using card1
- Added a column measuring interaction between C1 and C2

```
X_train['TransactionAmt_log'] = np.log(X_train['TransactionAmt'])
X_train['TransactionAmt_decimal'] = X_train['TransactionAmt'] % 1
X_train['TransactionAmt_decimal_len'] = X_train['TransactionAmt'].apply(lambda x: len(str(x).split('.')[1]) if '.' in str(x) else 0)
X_test['TransactionAmt_log'] = np.log(X_test['TransactionAmt'])
X_test['TransactionAmt_decimal'] = X_test['TransactionAmt'] % 1
X_test['TransactionAmt_decimal_len'] = X_test['TransactionAmt'].apply(lambda x: len(str(x).split('.')[1]) if '.' in str(x) else 0)
X_train['TransactionHour'] = (X_train['TransactionDT'] // 3600) % 24
X_test['TransactionHour'] = (X_test['TransactionDT'] // 3600) % 24
X_train['TransactionDay'] = (X_train['TransactionDT'] // (3600 * 24)) % 7
X_test['TransactionDay'] = (X_test['TransactionDT'] // (3600 * 24)) % 7

# Standardized transaction amount by card1
by
card1_mean_test = X_test.groupby('card1')['TransactionAmt'].transform('mean')
X_test['TransactionAmt_card1_mean'] = (X_test['TransactionAmt'] - card1_mean_test) / card1_mean_test

# Interaction between C1 and C2
X_train['C1_C2_interaction'] = X_train['C1'] * X_train['C2']
X_test['C1_C2_interaction'] = X_test['C1'] * X_test['C2']
```

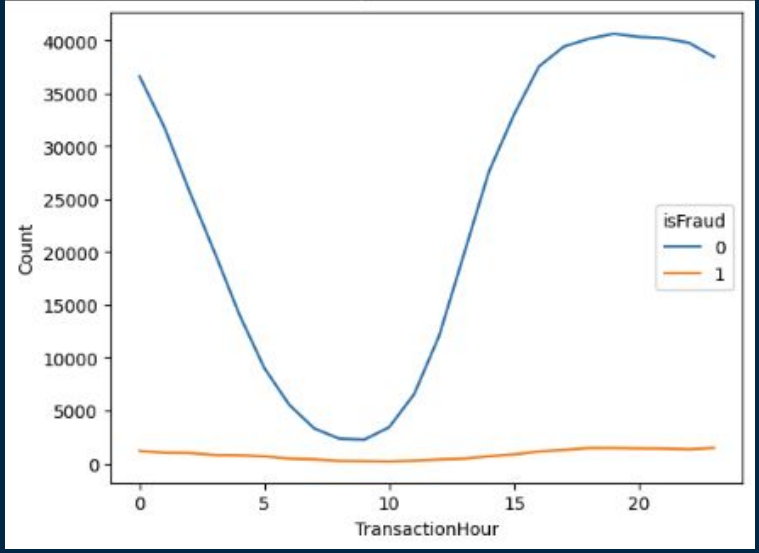
Feature Visualizations

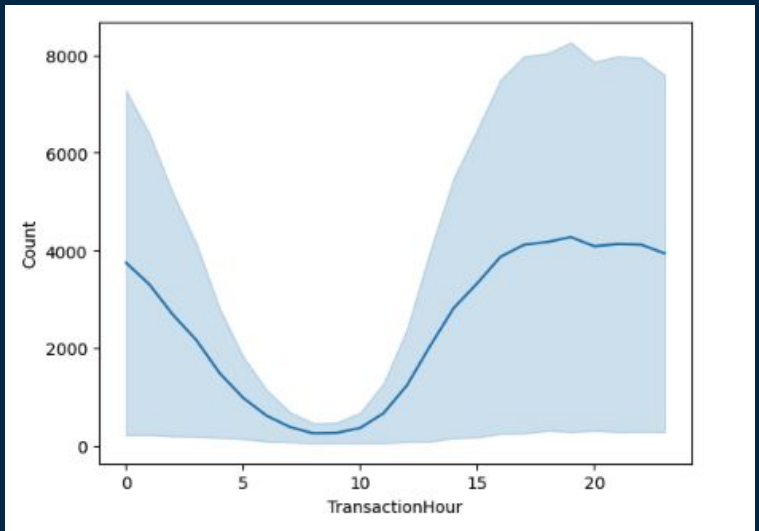


- Looked into correlations between a few of the most important features of our training dataset
- Interesting, but potentially not very useful correlation between `TransactionDT` and `TransactionID`
- Not much of interest to be seen → avenue of approach isn't successful

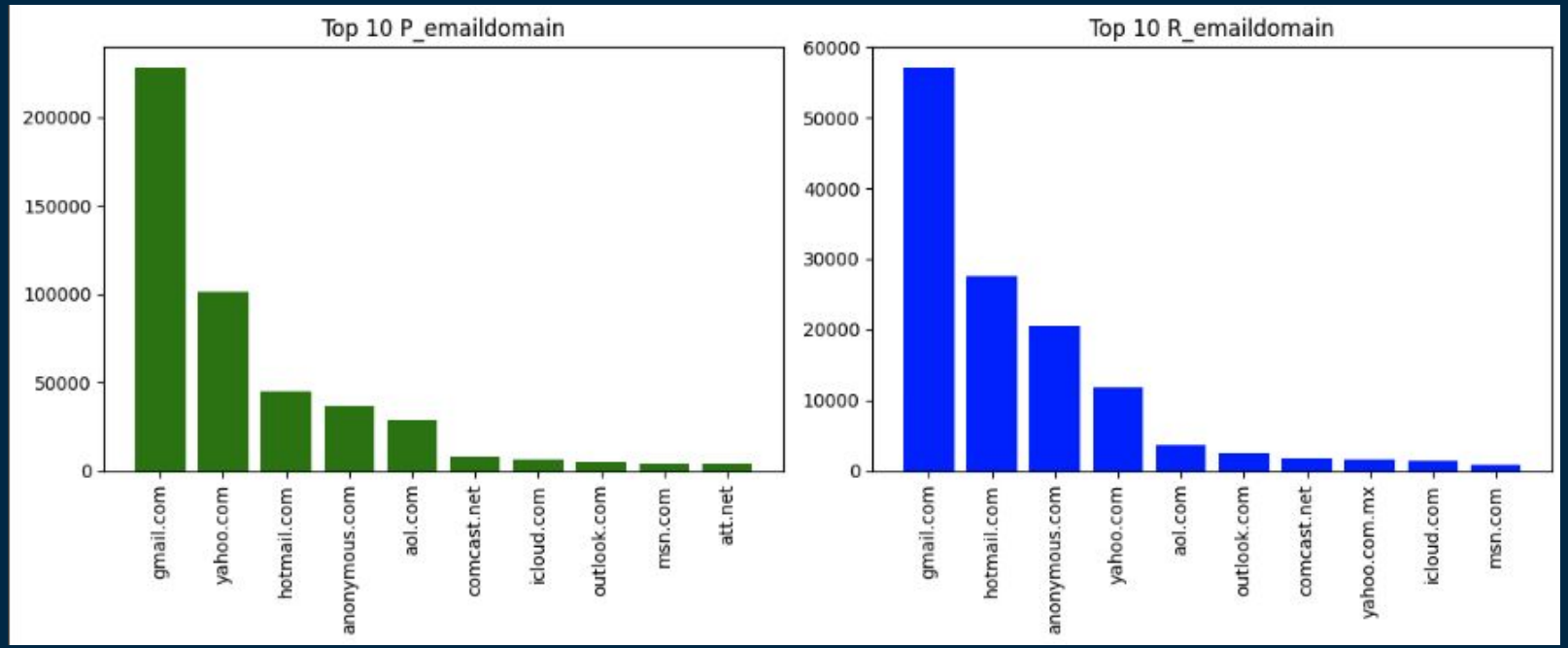
Feature Visualizations

- Inspected fraudulent and non fraudulent transactions over time
- Expected: A dip over what is presumably night time (before 10 AM) and increases throughout the day
- Highlights stark contrast between # of fraudulent and # of authentic transactions



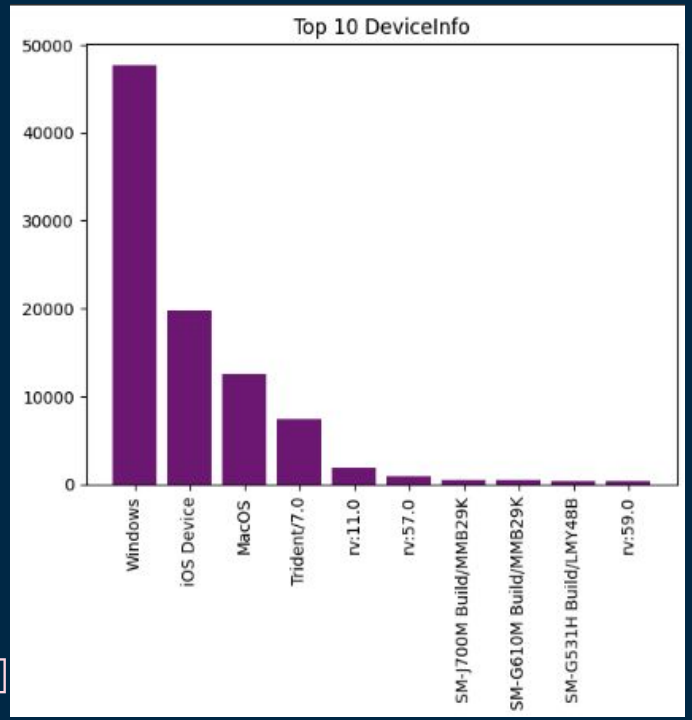


Feature Visualizations

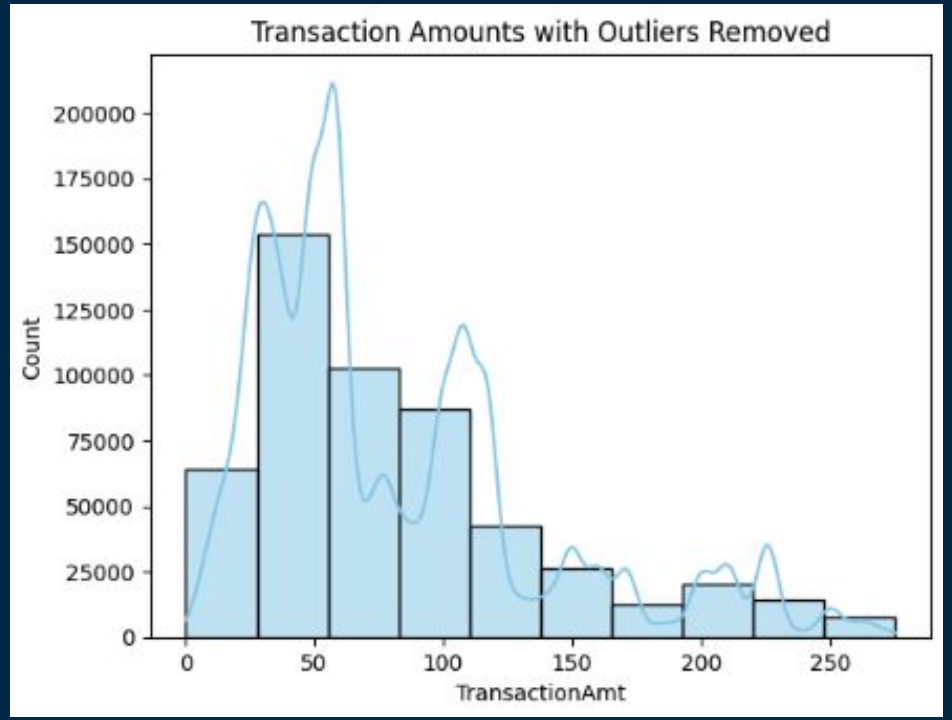


Feature Visualizations

Most common `DeviceInfo`



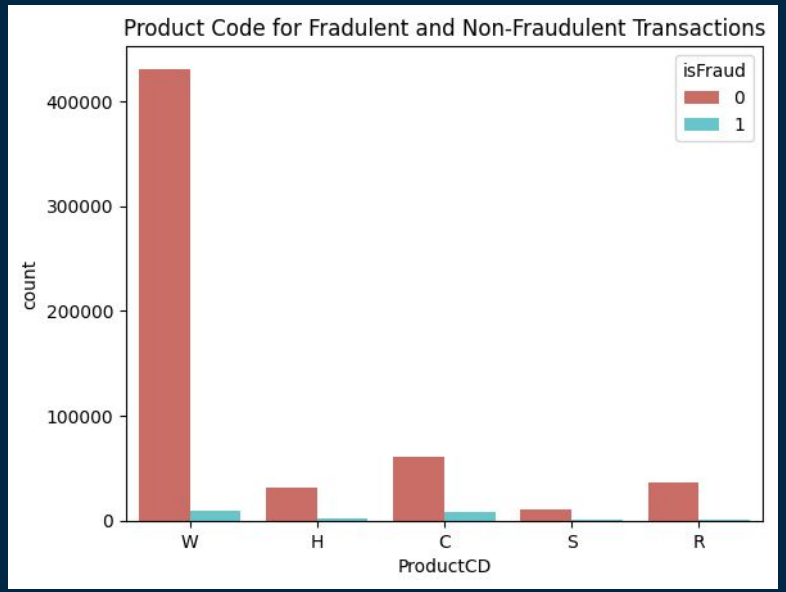
Spread of `TransactionAmt`



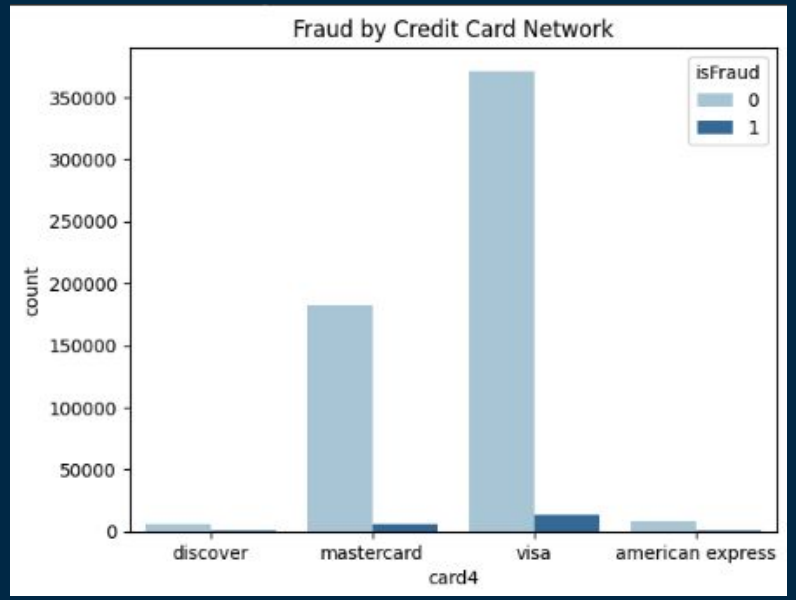
* removed top 10% of transactions

Seeking Answers to Other Questions

Are Different Kinds of Products More Likely to Appear in Fraudulent Transactions?



Are Different Card Networks More Likely to Have Fraudulent Transactions?



Alternative Datasets

- Incorporate identity dataset; minimal feature engineering
- Stricter Condition: no more than 5% of missing values
- Merging the transaction and identity dataset together → larger dataset with more variables to work with
- With a fraudulent transaction ratio of 7.8%, the dataset is also hopefully more resistant due to a larger sample size

	TransactionID	isFraud	TransactionDT	TransactionAmt	card1	card2	card3	card5	C1	C2	...	id_12	id_15	id_28	id_29	id_31	id_35	id_36	id_37
0	2987004	0	86506	50.000	4497	514.0	150.0	102.0	1.0	1.0	...	NotFound	New	New	NotFound	samsung browser 6.2	T	F	T
1	2987008	0	86535	15.000	2803	100.0	150.0	226.0	1.0	1.0	...	NotFound	New	New	NotFound	mobile safari 11.0	T	F	F
2	2987010	0	86549	75.887	16496	352.0	117.0	134.0	1.0	4.0	...	NotFound	Found	Found	Found	chrome 62.0	F	F	T
3	2987011	0	86555	16.495	4461	375.0	185.0	224.0	1.0	1.0	...	NotFound	New	New	NotFound	chrome 62.0	F	F	T
4	2987016	0	86620	30.000	1790	555.0	150.0	226.0	1.0	1.0	...	NotFound	Found	Found	Found	chrome 62.0	T	F	T

Additional Preprocessing

- Filling in NaN values
- One hot encoding categorical variable
- Nominally encoding features with too many categories
- Dropping unnecessary columns

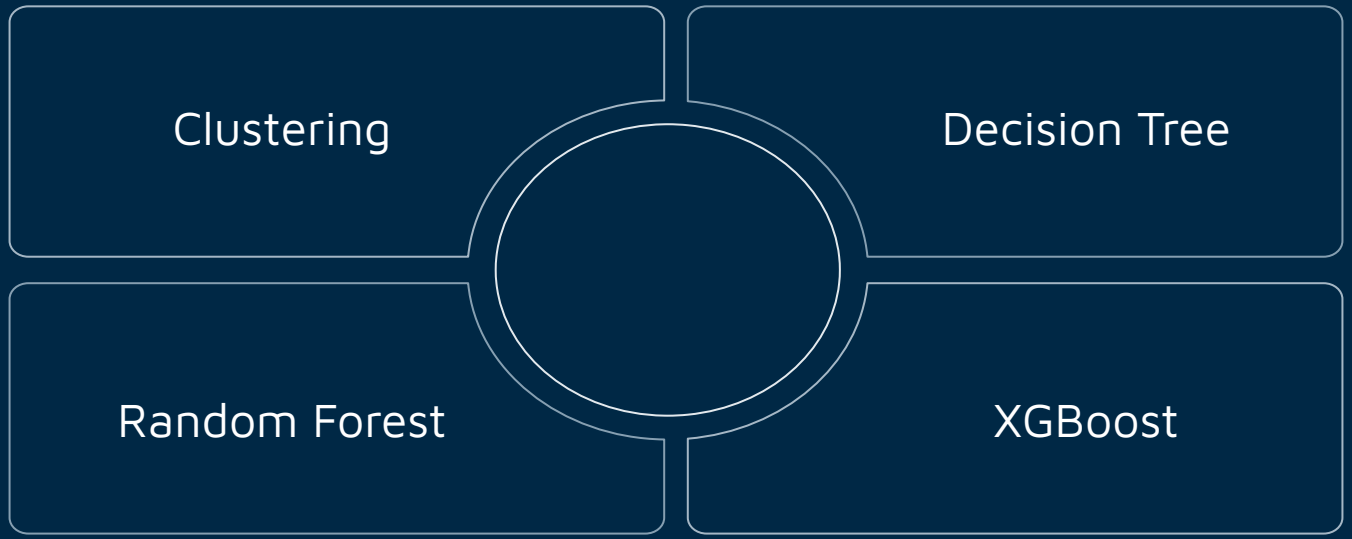
ProductCD_C	ProductCD_H	ProductCD_R	ProductCD_S	card4_N/A	card4_american express	card4_discover	card4_mastercard	card4_visa
0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0
1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0
...	...	...	...	...	...	...	...	...
1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0

id_31
samsung browser 6.2
mobile safari 11.0
chrome 62.0
chrome 62.0
chrome 62.0
...
chrome 66.0 for android
chrome 55.0 for android
chrome 65.0 for android
chrome 66.0
chrome 66.0 for android

03

Our Methods

Data Training - Models Explored



Clustering

- 2 possible outcomes
  - Fraud
  - Not Fraud
- Clustering is considered to see if there are any obvious groups that can make classification simpler
  - Straightforward and simple

```
X_train1 = X_train.copy()
X_test1 = X_test.copy()
y_train1 = y_train.copy()
y_test1 = y_test.copy()

kmeans_model = KMeans(n_clusters = 2,random_state = 42)
X_train1['cluster'] = kmeans_model.fit_predict(X_train1)
X_test1['cluster'] = kmeans_model.predict(X_test1)


cluster_pred = X_test1['cluster']
accuracy_score(y_test1, cluster_pred)
```

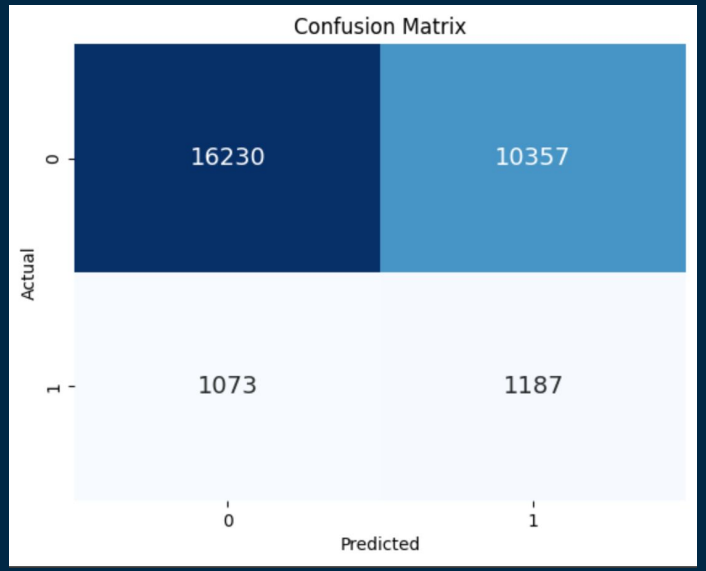
```
#Separating ground truth from dataset
X_set = training_data_features.drop(columns = 'isFraud')
y_set = training_data_features['isFraud']

#Splitting into training and testing set
X_train,X_test,y_train,y_test = split(X_set,y_set,test_size = 0.2,random_state = 42)
```

Clustering

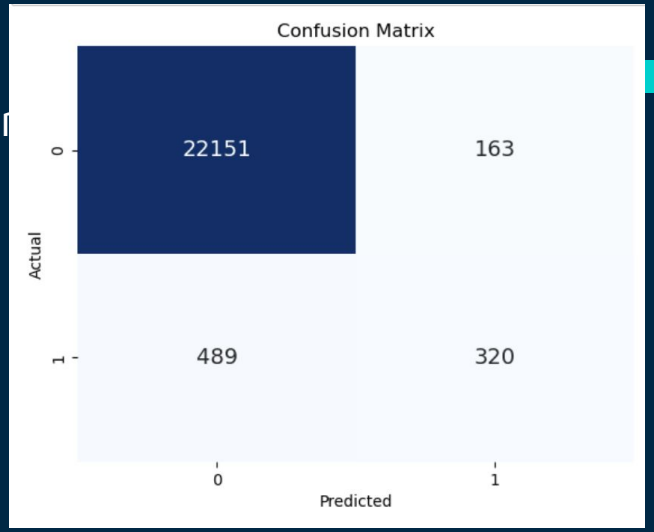
- Accuracy of 60%
  - Model isn't very impressive
  - Recall rate is barely above 50%

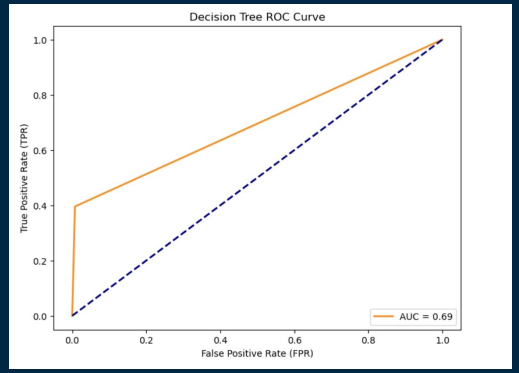
	precision	recall	f1-score	support
0	0.94	0.61	0.74	26587
1	0.10	0.53	0.17	2260
accuracy			0.60	28847
macro avg	0.52	0.57	0.46	28847
weighted avg	0.87	0.60	0.70	28847



Decision Tree

- Utilized to take advantage of its tree structur
  - Especially when a bank may need to explain to its customers why their card got declined
- Disadvantage:
  - May cause overfitting

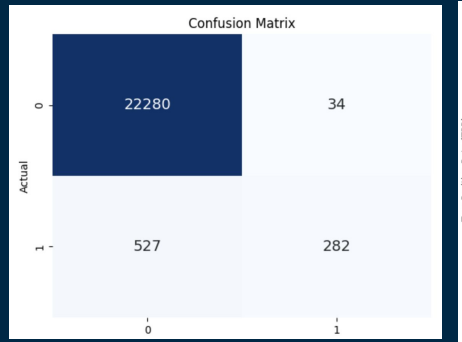


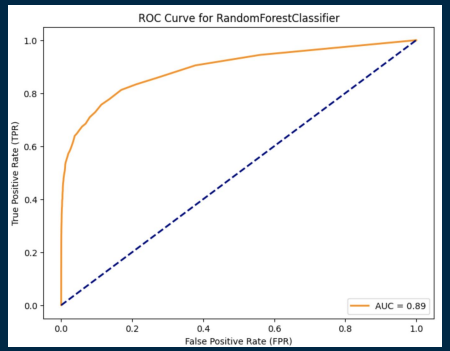


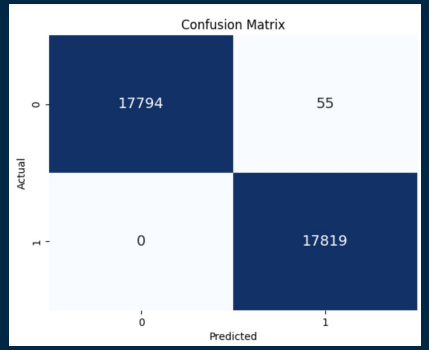
	precision	recall	f1-score	support
0	0.98	0.99	0.99	22314
1	0.66	0.40	0.50	809
accuracy			0.97	23123
macro avg	0.82	0.69	0.74	23123
weighted avg	0.97	0.97	0.97	23123

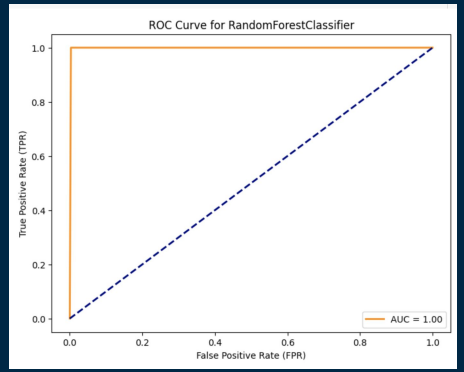
Random Forest

	precision	recall	f1-score	support
0	0.98	1.00	0.99	22314
1	0.89	0.35	0.50	809
accuracy			0.98	23123
macro avg	0.93	0.67	0.74	23123
weighted avg	0.97	0.98	0.97	23123









- Random Forest builds and merges multiple decision trees, generally stable prediction and good in classification
- Predicting fraud: Correct 89% of the time
- Capture 35% of actual fraud cases
- To improve model, also employed techniques such as oversampling minority class
  - Can lead to overfitting and overly optimistic evaluation metrics since fit on data not seen in true distribution of fraud

XGBoost Overview

- Ensemble learning algorithm that takes advantage of boosting
  - Uses errors in old models to improve upon newer ones
- Allows for different weights to classes
  - Give more emphasis to minority class during training → works particularly well for imbalanced datasets (like ours)

XGBoost Results

- Using XGBoost along with a dataset with a higher percentage of fraud cases would hopefully account for the relatively smaller occurrence of fraud cases
- Compared to other models shown, XGBoost takes significantly longer to process the data

```
# Set class weights
class_weights = {0: 1, 1: 10}  # Class 0: 1, Class 1: 10

# Set "scale_pos_weight" based on class balance
pos_weight = sum(class_weights.values()) / sum(class_weights.keys())


grid_values_xgb = {'learning_rate': [0.05, 0.1, 0.2],
                   'max_depth': [4,5,6],
                   'subsample': [0.5],
                   'colsample_bytree': [0.8,1],
                   'lambda': [1,1.1,1.2],
                   'alpha': [0,0.1,0.2],
                   'n_estimators': [100],
                   'scale_pos_weight':[pos_weight],
                   'random_state': [42]}

xgb_model = XGBClassifier()
xgb_cv = GridSearchCV(xgb_model, param_grid=grid_values_xgb, scoring='recall', cv=5, verbose=1)
xgb_cv.fit(X_train2, y_train2)
```

```
xgb_pred = xgb_cv.predict(X_test2)
xgb_cm = confusion_matrix(y_test2, xgb_pred)
xgb_acc = accuracy_score(y_test2, xgb_pred)

print ("Confusion Matrix: \n", xgb_cm)
print(classification_report(y_test2,xgb_pred))

xgb_TPR =  xgb_cm.ravel()[3] / (xgb_cm.ravel()[3] + xgb_cm.ravel()[2]) # TPR = TP/P = TP/(TP+FN)
xgb_FPR =  xgb_cm.ravel()[1] / (xgb_cm.ravel()[1] + xgb_cm.ravel()[0]) # FPR = FP/N = FP/(FP+TN)
xgb_PRE = xgb_cm.ravel()[3] / (xgb_cm.ravel()[3] + xgb_cm.ravel()[1]) # PRE = TP/(TP+FP)
```

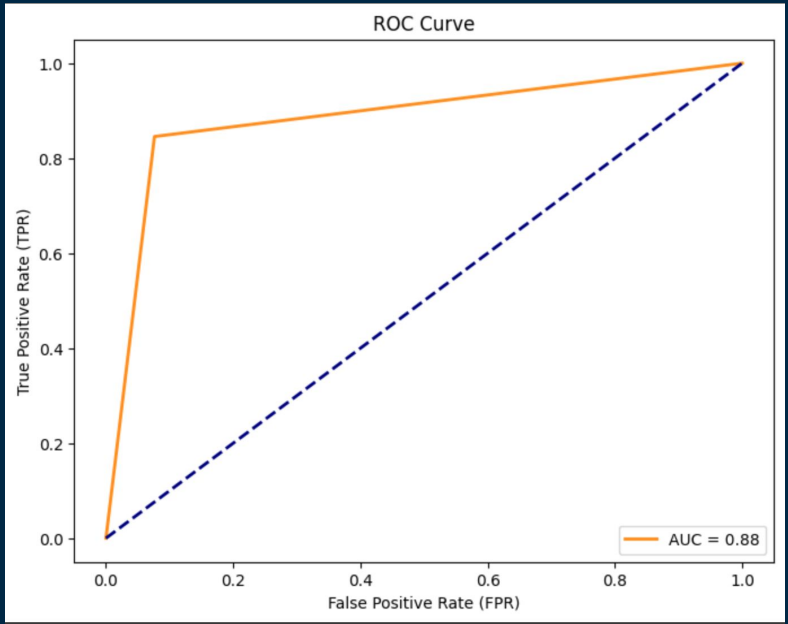
XGBoost Results

- In response, the model provides good results
  - Gives a recall rate of 0.85

```
Fitting 5 folds for each of 162 candidates, totalling 810 fits
time: 1276.99 s
Confusion Matrix:
 [[24560  2027]
 [  349  1911]]
              precision    recall  f1-score   support

           0       0.99      0.92      0.95     26587
           1       0.49      0.85      0.62      2260

    accuracy                           0.92     28847
   macro avg       0.74      0.88      0.79     28847
weighted avg       0.95      0.92      0.93     28847
```



04

Comparison of Results & Analysis

Evaluating Models

- Although a frequent occurrence, the rarity of credit card frauds vs total number of transactions makes it extremely difficult to develop a good model
  - Federal Reserve states that < 1% of all credit card transactions are fraudulent
  - Our dataset is also similarly extremely unbalanced
- Accuracy is generally an important performance metric, but not particularly in this case with unbalanced datasets
  - Crucial that we prioritize a model which provides the lowest false negative rate, even if it means an increased false positive rate
  - A metric like recall (TP / (TP + FN)) is more beneficial
    - Preferable for someone's transaction to be falsely flagged than for a fraudulent transaction to be missed

Comparing Models and Performance

- Ultimately, we saw that models like decision tree, clustering, and random forest struggle to identify actual fraud cases because of the dataset imbalance
  - Clustering performed poorly
    - Identified some fraud (10% precision) but only able to capture about 50% of actual fraud cases
  - Decision tree
    - Identified some fraud (66% precision) but only able to capture about 40% of actual fraud cases
  - Random forest performed well
    - Identified fraud (89% precision) but only able to capture 35% of actual fraud cases
  - XGBoost performed very well
    - Captured 85% of actual fraud cases

Best Model?

- Techniques such as oversampling the minority class in Random Forest greatly increased performance on class 1, but this presents other issues
- XGBoost allows for more focus on minority class instances that are misclassified
  - Builds sequential series of weak learners one after the other
  - Can focus on correcting the errors of previous trees
- Thus, **XGBoost** performed the best with precision and recall on class 1 of 0.5 and 0.85 respectively

Project Implications

While the project showcases some good results, it is still not yet ready to be applicable into the real world. Notably, the recall and precision rate is still insufficient.

A recall rate of 0.85 isn't bad but we would still like it to at least hit 0.9 since it is very important to correctly identify actual fraud cases, due to their time-sensitive nature.

Additionally, a precision of 0.5 is far from ideal. While we do prioritize the recall rate, ideally half of our fraud reports shouldn't be duds as they will annoy our clients and might present a bad image.

As a step forward, we would like to continue testing our dataset on additional models and possibly conducting more feature engineering with the means to improve our recall rate and precision.

Thank You!

Any Questions?